Zephyr Project

IT-Schulungen.com
Technology & Transformation

Vorkenntnisse und Start-Check

Technik-Check für native_sim

1.02

Der Start-Check in fünf Schritten

Vom persönlichen Ausgangspunkt bis zum lauffähigen Smoke-Test

01

Profil

C, Linux, Embedded und RTOS

02

Host

Linux, WSL 2 oder VM

03

Werkzeuge

Python, west, CMake und Ninja

04

Workspace

Version und Module prüfen

05

Ziel

native_sim bauen und starten

Probleme werden sichtbar gesammelt und gezielt priorisiert.

Kurze Selbsteinschätzung

C und Embedded	Syntax, Pointer, Build-Prozess und Mikrocontroller
Linux und Shell	Terminal, Pfade, Prozesse und Umgebungsvariablen
Git und CMake	Repository-Status, Branches und Build-Konfiguration
RTOS	Threads, Scheduling, Timing und Synchronisation
Zephyr	west, Kconfig, Devicetree und Board-Ziele

Markieren Sie je Bereich: sicher, bekannt oder neu.

Host-Umgebung gemeinsam prüfen

LINUX

Nativ oder VM

$ uname -s

$ pwd

$ whoami

WSL 2

Windows-Integration

wsl --status

wsl -l -v

Linux-Home verwenden

LINUX-VM

Vorbereitetes System

Netzwerk verfügbar

Genügend Speicher

Snapshot vorhanden

Entscheidend ist eine reproduzierbare Linux-Shell – nicht das Host-Betriebssystem.

Toolchain prüfen

TERMINAL

```
$ python3 --version
```

$ west --version	Workspace- und Build-Werkzeug
$ cmake --version	Build-Konfiguration
$ ninja --version	schneller Build-Executor
$ cc --version	Host-Compiler für native_sim
$ which west	Pfad zur aktiven Python-Umgebung

Alle Werkzeuge müssen aus der vorgesehenen Trainingsumgebung aufgerufen werden.

Workspace und Zephyr-Zustand

TERMINAL

```
$ west topdir
```

$ west topdir	zeigt Root des aktiven Workspaces
$ git -C zephyr describe --tags --always	prüft die ausgecheckte Zephyr-Version
$ west list	zeigt Projekte und Module
$ west status	macht lokale Änderungen sichtbar

Version, Workspace und Module müssen vor dem ersten Build zusammenpassen.

Welches native_sim-Ziel verwenden wir?

01	`Standard`	Bevorzugtes Ziel: native_sim
02	`64-Bit-Fallback`	native_sim/native/64 bei fehlenden 32-Bit-Hostbibliotheken
03	`Entscheidung`	Standard testen; nur bei Hostabhängigkeit wechseln

Das Ziel ändert die Hostausführung, nicht die Lernziele der Anwendung.

Smoke-Test: Build und Start

TERMINAL

```
$ west build -b native_sim zephyr/samples/hello_world
$ ./build/zephyr/zephyr.exe
Hello World! native_sim
```

01 Erwartet

Build endet ohne Fehler
Ausgabe erscheint im Terminal

02 Wenn nicht

Fehlermeldung sichern
Ziel, Toolchain und Pfade prüfen

Smoke-Test erfolgreich: Build, Prozessstart und Konsolenausgabe funktionieren.

Start-Check abgeschlossen

ARBEITSPLATZ

Umgebung bereit

Linux-Shell verfügbar

Python-Umgebung aktiv

Host-Tools erreichbar

WORKSPACE

Zephyr konsistent

west topdir erfolgreich

Version bekannt

Module geprüft

SMOKE-TEST

Ausführung bestätigt

Ziel ausgewählt

Build erfolgreich

Ausgabe sichtbar

Offene Punkte kommen mit Fehlermeldung und Verantwortlichkeit auf die gemeinsame Problemliste.